



VERSIONAMENTO E METODOLOGIAS ÁGEIS



SCTEC

SUMÁRIO

VERSIONAMENTO E METODOLOGIAS ÁGEIS

Apresentação	3
Introdução ao Git	4
Centralizado vs. distribuído	4
Git internamente	6
Integridade criptográfica: O <i>hash</i> SHA-1	7
O diretório git	8
A arquitetura de trabalho no Git	9
Arquitetura de ramificação	10
O que é uma ramificação?	11
GitFlow	11
Ramificação eterna	11
Ramificação temporária	12
Desenvolvimento <i>trunk-based</i>	12
Versionamento semântico	13
Ecossistema GitHub e a revisão de códigos	14
Topologia de redes	15
<i>Pull request</i>	17
Revisão de códigos	18
Integração contínua	18
Regras de proteção de ramificações	19
PraticAI: Do código legado ao versionamento colaborativo	20
Engenharia ágil	21
Do modelo cascata ao ágil	21
Métricas de engenharia: como medir o sucesso?	23
<i>Scrum</i>	23
<i>Kanban</i>	24
Lei de Conway: a estrutura do time define a arquitetura	26
Três vias do DevOps	27
PraticAI: Limitando o WIP na vida real	28
O fluxo unificado: GitFlow + <i>scrum</i> + CI/CD	28
Referências	31

APRESENTAÇÃO

No desenvolvimento moderno, o código não é estático, ele é dinâmico e muda a cada minuto pelas mãos de dezenas de desenvolvedores. Sem uma gestão rigorosa, esse dinamismo vira caos. Dessa forma, este material não é apenas um guia de comandos, mas sim um mergulho na arquitetura de processos que sustentam grandes times de tecnologia.

Dominar o Git e as metodologias ágeis é um diferencial enorme. Enquanto o código resolve o problema do usuário, o versionamento e o processo resolvem os problemas do time. Sem eles, a escalabilidade é impossível.

Então, aqui será abordado desde a matemática criptográfica que garante a integridade do Git, passando pelas estratégias de ramificação (GitFlow) que organizam o trabalho paralelo, até os ritos ágeis (*scrum/kanban*) que conectam pessoas ao código. O objetivo é formar profissionais capazes de orquestrar o fluxo de valor do software, não apenas escrever funções.

Bons estudos!



INTRODUÇÃO AO GIT

Muitos desenvolvedores usam o Git como um simples botão de “salvar” anabolizado, memorizando comandos como *add*, *commit* e *push* sem entender o que realmente acontece nos bastidores.

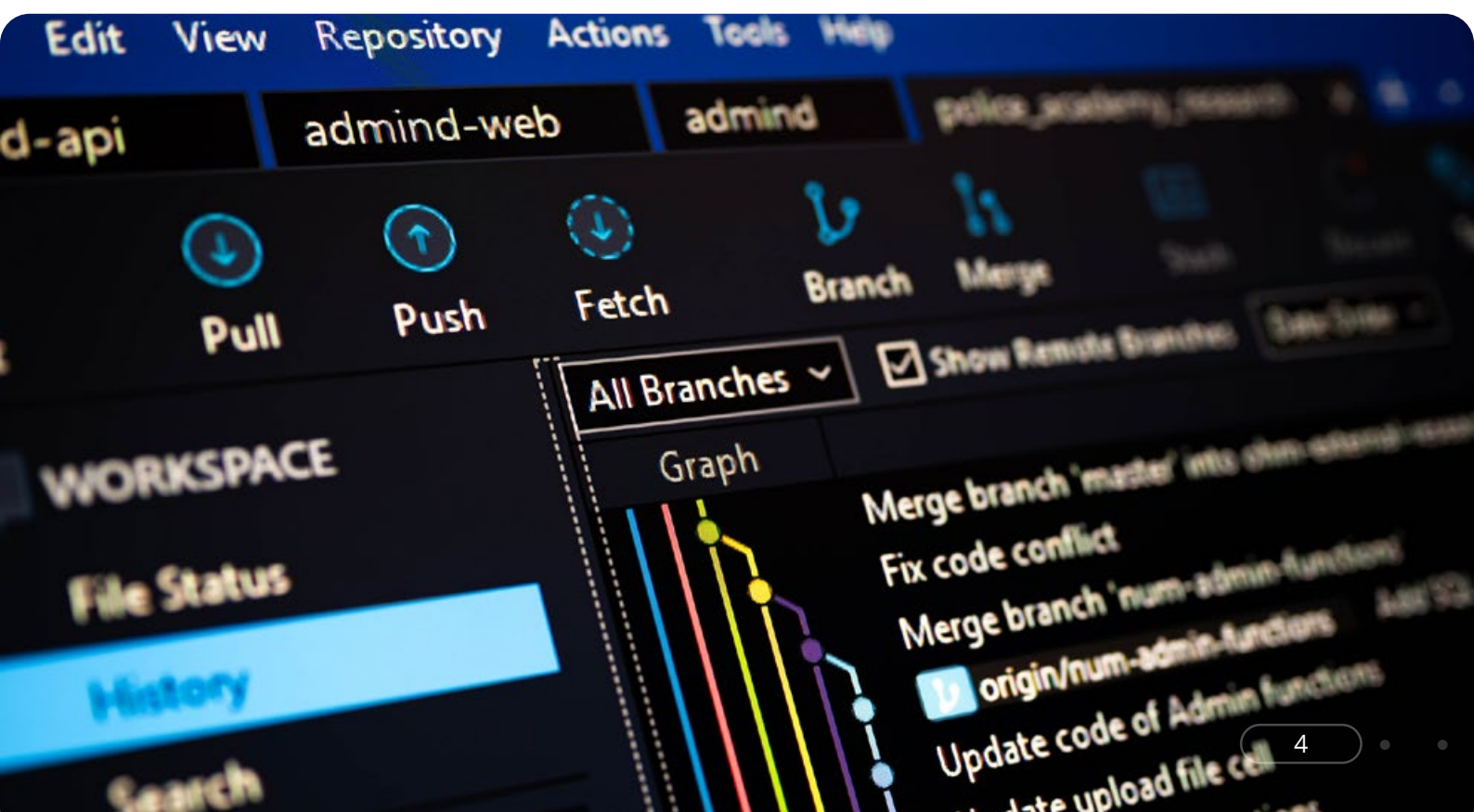
Para um engenheiro de software, essa visão superficial é perigosa. O Git não é apenas uma ferramenta de backup; é um sistema de arquivos distribuído, endereçável por conteúdo, e construído sobre uma estrutura de dados de grafos acíclicos.

Neste capítulo, a “caixa preta” da pasta **.git** vai ser aberta e você vai entender a matemática que torna o versionamento possível.

Centralizado vs. distribuído

Antes do Git se tornar o padrão da indústria, o desenvolvimento de software era dominado por sistemas de controle de versão centralizados (CVCS), como o *subversion* (SVN) e o CVS. Nesses sistemas, o modelo mental é baseado em um servidor único que detém a “verdade” e todo o histórico de versões. O desenvolvedor, em sua máquina local, realiza um checkout apenas da última versão dos arquivos (*snapshot* atual).

Isso cria uma dependência severa da rede: para ver o histórico de um arquivo, criar uma ramificação ou salvar uma alteração (*commit*), é obrigatório estar conectado ao servidor central. Se a rede cair ou o servidor corromper, o desenvolvimento para.



O Git, criado por Linus Torvalds em 2005 para gerenciar o desenvolvimento do kernel do Linux, popularizou a arquitetura distribuída (DVCS). A diferença é estrutural: quando você executa um **git clone**, não está apenas baixando os arquivos para trabalhar, e sim está copiando o banco de dados inteiro do projeto para o seu disco rígido. Cada clone é um repositório completo, contendo cada *commit*, cada ramificação e cada tag já criados desde o início do projeto.

Essa mudança traz três vantagens de engenharia fundamentais:

- 1. Independência e velocidade:** Quase todas as operações (**diff**, **log**, **commit**, **branch**, **merge**) ocorrem localmente. O Git não precisa “ir à rede” para calcular a diferença entre duas versões; ele apenas lê o seu disco. Isso torna operações que levavam segundos (ou minutos) no SVN em processos instantâneos no Git.
- 2. Resiliência:** Não existe um “ponto único de falha”. Se o servidor principal (como o GitHub) sair do ar ou perder dados, o repositório de qualquer desenvolvedor da equipe pode ser usado para restaurar o servidor completo, pois todos têm uma cópia integral dos dados.
- 3. Fluxo de trabalho não linear:** Como o custo de criar ramificações (*branches*) é local e irrisório, o Git encoraja a experimentação constante, algo que era proibitivo em sistemas centralizados devido à complexidade e à lentidão de gerenciar ramificações no servidor.

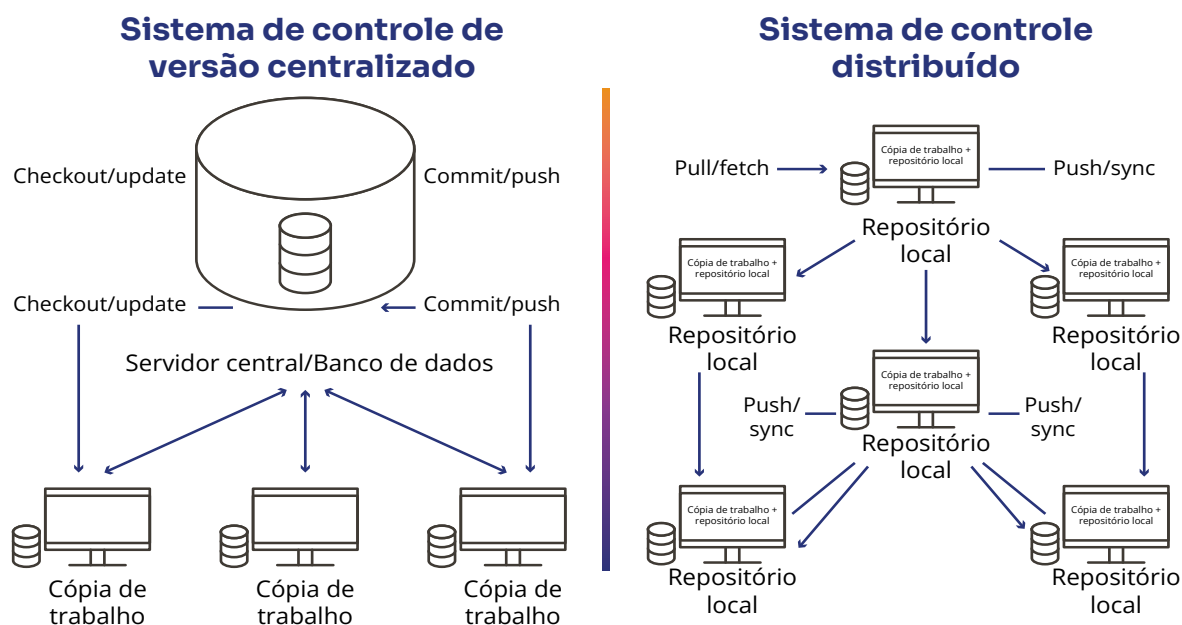


Figura 1 - Comparação de sistemas de controle centralizado e distribuído

Fonte - Da autora (2025)

Git internamente

Ao contrário do que a intuição sugere, o Git não armazena “diferenças” (deltas) entre arquivos por padrão. Ele armazena *snapshots* (fotografias) do projeto inteiro. Para entender isso, é preciso descer ao nível do *plumbing* (encanamento), ou seja, os comandos de baixo nível que manipulam o banco de dados de objetos do Git (Chacon; Straub, 2014).

O banco de dados do Git (localizado em **.git/objects**) é um armazenamento chave-valor simples, no qual a chave é um **hash SHA-1** de 40 caracteres e o valor é um dos três tipos de objetos fundamentais:

- **Blob (*binary large object*):** Armazena o conteúdo de um arquivo, mas não seu nome. Se você tiver dois arquivos com o mesmo texto em pastas diferentes, o Git armazenará apenas um *blob*, economizando espaço automaticamente. Você pode testar isso rodando:

```
echo "Ola" | git hash-object --stdin
```

- **Tree (árvore):** Representa um diretório. Ela mapeia nomes de arquivos para seus respectivos *blobs* e mapeia subdiretórios para outras *trees*. É ela que dá nome aos arquivos e define a estrutura de pastas.
- **Commit (nó):** É o objeto que dá contexto. Ele aponta para uma *tree* principal (o estado do projeto naquele momento), contém metadados (autor, data, mensagem) e, crucialmente, aponta para o *commit* pai.

Essa estrutura cria um grafo direcionado acíclico ou DAG (*directed acyclic graph*). O histórico do Git não é uma linha reta, é um grafo matemático onde cada *commit* é imutável. Assim, se você alterar uma vírgula em um arquivo antigo, o *hash* do *blob*, da *tree* e do *commit* também mudam, invalidando toda a cadeia subsequente. É isso que torna o Git à prova de adulteração.



Curiosidade curiosa

Se o Git salva *snapshots* inteiros, o repositório não fica gigante? Não, graças aos *packfiles*. O Git armazena os arquivos completos (*loose objects*) inicialmente para velocidade. Mas, periodicamente (ou quando você roda **git gc**), ele compacta esses objetos, buscando arquivos similares e armazenando apenas as diferenças binárias entre eles em um único arquivo comprimido **.pack**. É uma engenharia brilhante de otimização de espaço.

Integridade criptográfica: O hash SHA-1

A base de confiança do Git reside no algoritmo SHA-1 (*secure hash algorithm 1*). Cada arquivo, diretório ou *commit* no Git é identificado por um *checksum* de 40 caracteres hexadecimais, calculado a partir do seu conteúdo.

Isso significa que o Git é endereçável por conteúdo, de modo que o nome do arquivo no banco de dados interno é o próprio *hash* do seu conteúdo.

Por exemplo, se o arquivo **texto.txt** contiver a palavra “Olá”, o Git calculará o *hash* específico dessa *string* e salvará o arquivo com esse identificador. Caso você altere o conteúdo para “Olá, Mundo”, o *hash* mudará completamente, levando o Git a salvar um novo objeto independente.

Essa propriedade garante a integridade dos dados, tornando impossível perder informações durante a transmissão, ou ter um arquivo corrompido no disco sem que o sistema perceba, já que o *hash* calculado não corresponderia mais ao nome do objeto. Isso oferece a certeza de que o código baixado é idêntico ao que o autor escreveu.

O diretório .git

Quando você roda **git init** em uma pasta, tudo o que o Git cria é um subdiretório oculto chamado **.git**. Se você apagar essa pasta, perde todo o histórico, mas mantém os arquivos atuais. Nesse diretório, existem:

- **Pasta objects/:** Funciona como o banco de dados real em que ficam os *blobs*, *trees* e *commits*.
- **Pasta refs/:** Local em que ficam os ponteiros do sistema.
- **Branches e tags:** São arquivos de texto que contêm o *hash* de um *commit* específico.
- **Arquivo HEAD:** É um texto simples que atua como uma bússola, indicando ao Git onde você está posicionado no momento (geralmente apontando para uma *branch* como **refs/heads/main** por exemplo).
- **Arquivo config:** Armazena as configurações locais do projeto, como a URL do repositório remoto e suas credenciais de e-mail.

Saber disso permite, por exemplo, recuperar *commits* perdidos ou corrigir referências corrompidas editando arquivos manualmente em casos extremos de recuperação de desastre.



A arquitetura de trabalho no Git

Para gerenciar o grafo complexo de versões sem sobrecarregar o usuário, o Git abstrai o processo em três áreas de trabalho distintas, cuja compreensão é vital para evitar a perda de arquivos (Chacon; Straub, 2014).

O processo inicia-se no diretório de trabalho (*working directory*), que é a sua pasta atual, o “mundo real” do sistema operacional, no qual você edita, cria e deleta arquivos. É aqui que o comando “git status” verifica as mudanças em relação ao que já foi rastreado.

Antes de confirmar as alterações, os arquivos passam pelo *index* (*staging area*), que é uma área intermediária e invisível na qual você prepara a “foto” antes de tirá-la. Ao executar **git add arquivo.txt**, o Git captura o conteúdo do diretório de trabalho, cria um *blob* no banco de dados e atualiza o *index*, funcionando como um rascunho organizado do próximo passo.

Finalmente, as mudanças chegam ao repositório **.git**, que é o banco de dados histórico definitivo. Quando você roda **git commit**, o Git consolida o que está no *index*, cria os objetos *tree* e *commit* necessários e atualiza o ponteiro da ramificação atual (HEAD) para apontar para esse novo momento no tempo.

A ilustração a seguir demonstra visualmente como os arquivos “viajam” entre essas três zonas: nascem no diretório de trabalho, são preparados no *index*, por meio do comando **add**, e, finalmente, são eternizados no histórico do repositório via *commit*.

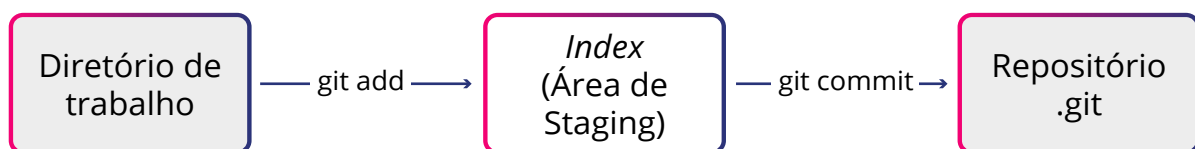


Figura 2 - As três áreas do Git pelas quais os arquivos passam

Fonte - Da autora (2025)



Atenção

Muitos iniciantes acham o **git add** redundante. Contudo, o *index* (*staging area*) permite atomicidade. Se você trabalhou em três bugs diferentes no mesmo arquivo, pode usar essa área para selecionar apenas as linhas do bug A (**git add -p**), fazer um *commit* focado e depois fazer o resto. Isso gera um histórico limpo e revisável, essencial para engenharia de software.

Neste capítulo, você aprendeu que o Git é uma estrutura de dados baseada em grafos (DAG) e *hashes* criptográficos (SHA-1) que garantem integridade, velocidade e distribuição. Além disso, pôde entender que os comandos **add** e **commit** são ferramentas para manipular *blobs* e *trees* entre três áreas de trabalho distintas dentro do diretório **.git**, onde toda a inteligência reside.

Com essa base sólida sobre como o Git armazena uma versão, você está pronto para o próximo desafio: como gerenciar múltiplas versões paralelas do mesmo projeto.

ARQUITETURA DE RAMIFICAÇÃO

No desenvolvimento profissional de software, a “linha do tempo” do projeto raramente é uma reta única e contínua. Ela se divide em múltiplos universos paralelos nos quais são criadas novas funcionalidades, corrigidos erros críticos e estabilizadas diferentes versões simultaneamente, tudo sem interferir no código que está em produção.

Para gerenciar essa complexidade, são utilizadas estratégias de ramificação (*branching models*). Não se trata apenas de executar comandos no terminal, mas de estabelecer acordos de arquitetura que definem rigorosamente como o código flui do computador do desenvolvedor até o servidor de produção.

```
1 # get random forest model
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.ensemble import RandomForestRegressor
5 from sklearn.metrics import mean_squared_error, r2_score
6
7 # load data from train.csv
8 train_df = pd.read_csv('data/train.csv')
9 train_df['target'] = train_df['target'].astype(int)
10
11 # split the data into training and testing sets
12 X_train, X_test, y_train, y_test = train_test_split(train_df, test_df,
```

O que é uma ramificação?

Muitos desenvolvedores hesitam em criar ramificações (*branches*) porque, em sistemas de controle de versão antigos como o SVN, isso significava copiar fisicamente todos os arquivos do projeto para um novo diretório, que era um processo lento e custoso para o armazenamento.

Contudo, no Git, a engenharia é radicalmente diferente e extremamente eficiente. Conforme explicado por Chacon e Straub (2014), um *branch* no Git não é uma cópia dos arquivos, mas tecnicamente um ponteiro móvel, sendo um arquivo de texto minúsculo (41 bytes) que contém o *hash* de um *commit* específico.

Quando você cria uma ramificação, o sistema cria apenas esse pequeno apontador, mantendo o custo da operação virtualmente zero (complexidade $O(1)$). Então, um outro ponteiro especial chamado **HEAD** indica qual *branch* está ativa no momento. Ao realizar um novo *commit*, o Git avança automaticamente o ponteiro da *branch* atual para o novo *snapshot*, deixando as outras paradas no tempo.

Essa leveza arquitetural permite e encoraja que profissionais criem ramificações para qualquer tarefa, seja para corrigir um erro ortográfico ou testar uma arquitetura complexa, garantindo o isolamento total das mudanças até que estejam prontas para integração.

GitFlow

Proposto por Vincent Driessen (2010), o GitFlow estabeleceu-se como um forte padrão da indústria para softwares com ciclos de lançamento agendados. Contudo, vale destacar que ele não é a única metodologia de versionamento existente, mas sim uma das diversas estratégias disponíveis para organizar o ciclo de vida do código, devendo ser escolhida de acordo com a necessidade do projeto. Sua arquitetura baseia-se na existência de dois tipos de ramificações: as eternas e as temporárias.

Ramificação eterna

As ramificações eternas, **main** (ou *master*) e **develop**, formam a espinha dorsal do projeto. A *main* armazena exclusivamente o código de produção, estável e pronto para o cliente, enquanto a *develop* serve como a linha de integração contínua em que todo código novo se encontra antes de ser polido.



Ramificação temporária

Ao redor dessa espinha dorsal, orbitam as ramificações temporárias que dão vida ao fluxo de trabalho. As de funcionalidade (**feature/***) nascem da *develop* e servem para o desenvolvimento isolado de novas tarefas, retornando à origem apenas quando finalizadas.

Quando a equipe decide que a *develop* acumulou funcionalidades suficientes para uma nova versão, cria-se uma ramificação de lançamento (**release/***). Esse ambiente atua como um “congelamento” do código, permitindo apenas correções de bugs finais e documentação, sem a interferência de novas funcionalidades.

Existe também a ramificação de correção (**hotfix/***). Se um erro grave for detectado em produção, ela é criada diretamente da *main*, permitindo uma correção imediata que, posteriormente, é fundida tanto na *main* quanto na *develop*, garantindo que o erro não retorne em versões futuras.

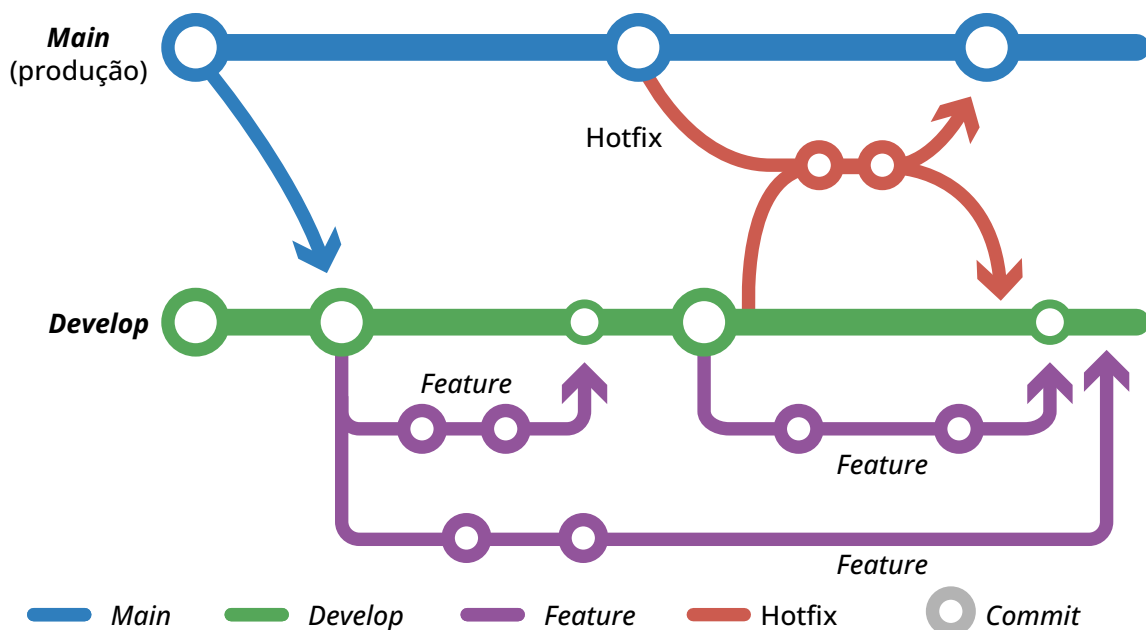


Figura 3 - Representação do GitFlow

Fonte - Da autora (2025)

Desenvolvimento trunk-based

Embora o GitFlow seja robusto, ele pode tornar-se burocrático e lento para equipes de alta performance que realizam *deploys* várias vezes ao dia, como Google ou Facebook.





Atenção

O excesso de ramificações de longa duração frequentemente resulta no temido “*merge hell*”, no qual a reconciliação de códigos que divergiram por semanas gera conflitos massivos e difíceis de resolver.

Para mitigar esse risco, a engenharia moderna tem adotado o desenvolvimento *trunk-based*. Nesse modelo, todos os desenvolvedores integram seu código diretamente na ramificação principal (chamada *trunk* ou *main*) ou utilizam ramificações de vida curtíssima, que duram menos de um dia.

O segredo para não quebrar a produção com código incompleto reside no uso de sinalizadores de funcionalidade (*feature flags*). O código novo é enviado para a *main*, mas fica envolvido em uma condicional lógica que o mantém inativo para o usuário final até que esteja pronto. Isso permite a integração contínua (CI), em que o código de toda a equipe é testado conjuntamente o tempo todo, eliminando surpresas desagradáveis nas vésperas do lançamento.



Dica

A escolha depende da natureza do seu produto. Utilize o **GitFlow** se você desenvolve softwares “encaixotados” ou aplicativos móveis, nos quais existe o conceito de versões fechadas (v1.0, v1.1). Opte pelo *trunk-based* se você gerencia uma aplicação web (SaaS) que exige atualizações constantes e *deploy* contínuo (DC).

Versionamento semântico

De nada adianta organizar o código internamente se o usuário não for comunicado do impacto de uma atualização. Para padronizar essa comunicação, a indústria adota o versionamento semântico (SemVer). Esse sistema utiliza um conjunto de três números (v1.2.3) no qual cada posição carrega um significado de engenharia preciso:

- **Major:** É o primeiro número e indica mudanças que quebram a compatibilidade (*breaking changes*), sinalizando que o usuário precisará adaptar seu sistema para atualizar.
- **Minor:** É o segundo número e representa a adição de novas funcionalidades que são totalmente compatíveis com a versão anterior.
- **Patch:** É o terceiro número e é reservado para correções de bugs simples que não alteram a API.



Dica

No Git, o recurso de tags (`git tag v1.0.0`) é utilizado para marcar, como uma etiqueta imutável, o *commit* exato na *main* que corresponde a esse lançamento, garantindo a rastreabilidade histórica do software.

As ramificações são os universos paralelos da engenharia de software. Você aprendeu que, tecnicamente, são apenas ponteiros, o que permite usar estratégias robustas como o GitFlow (para controle e lançamentos formais) ou o *trunk-based* (para velocidade e CI/CD). Você também aprendeu que o SemVer é a linguagem usada para comunicar mudanças ao mundo.

Agora que você já sabe organizar o código, é preciso aprender a colaborar e garantir que o código feito por colegas não tenha bugs antes de aceitá-los.

ECOSSISTEMA GITHUB E A REVISÃO DE CÓDIGOS

O GitHub (ou seus pares como GitLab e Azure DevOps) não deve ser encarado apenas como um local para armazenar backup de código na nuvem. Na engenharia de software moderna, ele atua como o sistema nervoso central do projeto, orquestrando a colaboração, a qualidade e a segurança do software. É nesse ambiente que transformamos o ato solitário de programar em um processo social e auditável de construção coletiva.

Topologia de redes

Para compreender como dezenas de engenheiros colaboram sem sobrescrever o trabalho uns dos outros, é fundamental entender a topologia de rede do Git, que difere radicalmente dos sistemas centralizados antigos.

No seu computador, o repositório é soberano, mas para colaborar, ele precisa conversar com outros repositórios. Esses pares são chamados de remotos (*remotes*). Tecnicamente, eles são apenas uma referência no arquivo de configuração (**.git/config**) que mapeia um nome curto como **origin** para uma URL complexa.



Quando você clona um projeto, o Git cria automaticamente o remoto *origin* apontando para a fonte. Contudo, em projetos de código aberto ou em grandes empresas, a arquitetura exige um nível adicional de direção conhecido como *fork*.

Um *fork* não é um comando nativo do Git, mas uma funcionalidade da plataforma (GitHub) que cria uma cópia completa do repositório no servidor, sob sua conta pessoal. Isso estabelece uma topologia triangular: o repositório original (frequentemente chamado de *upstream*), o seu *fork* no servidor (seu *origin*) e o seu clone local.

Essa distinção é vital para o fluxo de trabalho. Você não tem permissão para escrever diretamente no *upstream* da empresa, então você envia seu código para o seu *origin* e, de lá, solicita que os mantenedores do *upstream* aceitem suas alterações. Essa mecânica garante que o repositório principal permaneça limpo e protegido, recebendo apenas o código que passou por um processo de curadoria.

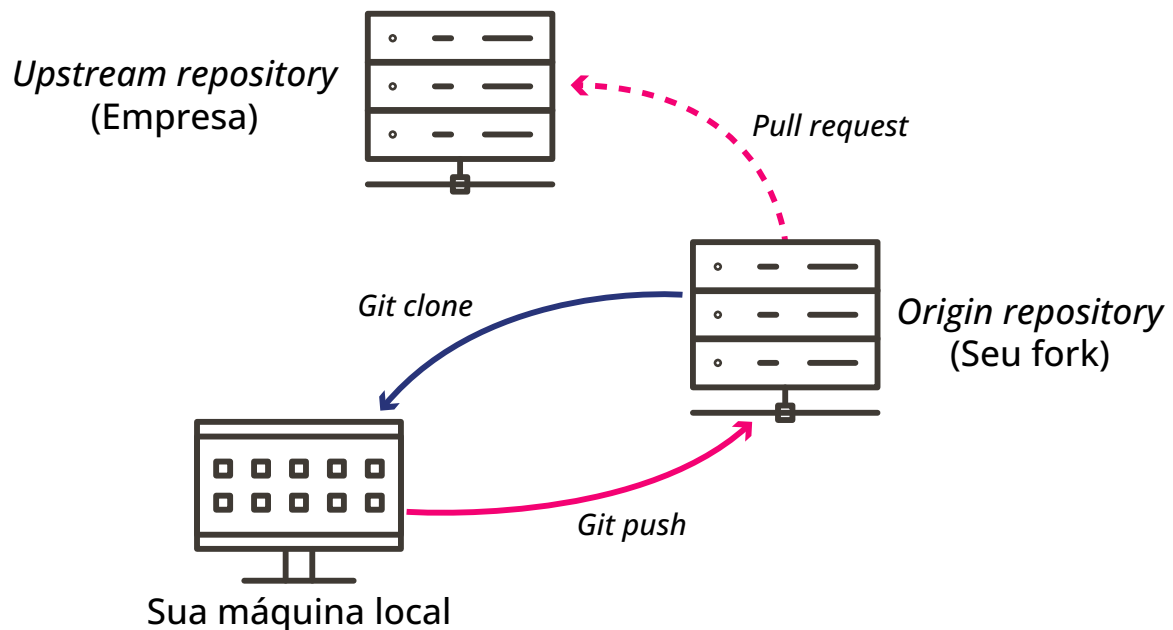


Figura 4 - Fluxo de trabalho usando um *fork* no GitHub

Fonte - Da autora (2025)

A sincronização nesse ambiente distribuído exige precisão no uso dos comandos. Muitos desenvolvedores confundem **git fetch** com **git pull**, o que pode levar a estados inconsistentes. Segundo Chacon e Straub (2014), a diferença é:

- **Git fetch:** É a operação segura que vai até o remoto, baixa todos os dados e referências novas para o seu banco de dados local, mas não toca nos seus arquivos de trabalho. Ele apenas atualiza o seu conhecimento sobre o estado do mundo.
- **Git pull:** É um comando macro que executa um **fetch** seguido imediatamente de um **merge**. Embora conveniente, ele pode ser perigoso se você não entender o que está sendo mesclado.



Dica

Engenheiros experientes frequentemente preferem buscar as alterações primeiro, inspecioná-las e só então decidir como integrá-las.

Pull request

A peça central da colaboração no ecossistema GitHub é o **pull request** (PR). Apesar do nome, que significa “pedido para puxar”, o PR deve ser entendido como um fórum de discussão sobre uma proposta de mudança. Ele representa o momento em que o código deixa de ser um rascunho privado e se submete ao escrutínio da equipe.



Atenção

Na engenharia de software, o PR atua como um portão de qualidade. Nenhum código deve entrar na ramificação principal sem passar por esse processo.

Segundo as práticas de engenharia do Google (202-), um PR bem estruturado deve contar uma história. Assim, não deve ter apenas linhas de código, mas explicar o “porquê” da mudança, o contexto do problema resolvido e os riscos envolvidos.

É dentro da interface do PR que ocorre a revisão do código, pois ali o GitHub apresenta o *diff* (a diferença visual entre o que era e o que será), permitindo que revisores comentem em linhas específicas. Essa granularidade transforma a revisão em uma conversa técnica precisa, por meio da qual podem ser resolvidas as dúvidas arquitetônicas antes que se tornem dívidas técnicas.

Além disso, o PR é o ponto de integração com sistemas automáticos: ele só permite o botão de fusão (**merge**) ficar verde se todas as verificações de segurança e testes passarem, garantindo que a ramificação principal nunca quebre.



Revisão de códigos

A revisão de código é, segundo as práticas de engenharia do Google (202-), fundamental não apenas para encontrar erros, mas para compartilhar conhecimento e manter a consistência da base de código. Quando um engenheiro sênior revisa o código de um júnior, ou vice-versa, ocorre uma transferência de conhecimento que nenhuma documentação consegue replicar.

Dessa forma, o revisor não deve agir como um corretor ortográfico (ferramentas automáticas, como *linters*, já fazem isso), e sim focar na lógica, na arquitetura, na segurança e na legibilidade. Durante a revisão, deve-se questionar se o código resolve o problema proposto, se ele introduz complexidade desnecessária e se possui testes adequados.

Forsgren, Humble e Kim (2018) demonstram estatisticamente que equipes que realizam revisões de código pequenas e frequentes têm maior estabilidade e menor taxa de falhas.



Atenção

Contudo, a revisão também exige *soft skills*: comentários devem ser sobre o código, nunca sobre a pessoa. Em vez de dizer “você errou aqui”, um engenheiro colaborativo diz “este trecho pode causar um problema de performance se a lista for grande, que tal usarmos outra abordagem?”. Esse tom construtivo é o que diferencia times de alta performance de ambientes tóxicos.

Integração contínua

Se a revisão cuida da qualidade lógica e arquitetural, a integração contínua (CI) cuida da integridade funcional. O conceito de CI é que todo código integrado à base principal deve ser verificável automaticamente (Humble; Farley, 2010). Ela é operacionalizada por ferramentas como o GitHub Actions.

No ecossistema GitHub, isso é configurado através de arquivos YAML (como **.github/workflows/main.yml**), que são receitas que dizem à plataforma o que fazer toda vez que um evento ocorre (como um *push* ou a abertura de um PR).

Quando você envia código, o GitHub aloca um servidor virtual (chamado de *runner*), baixa seu código, instala as dependências e executa uma bateria de testes automatizados. Se um único teste falhar, o sistema marca o PR com um “X” vermelho e bloqueia a fusão. Isso dá segurança para o time, pois é possível refatorar sistemas complexos com a confiança de que, se algo antigo quebrar, o robô avisará antes que o erro chegue ao cliente.

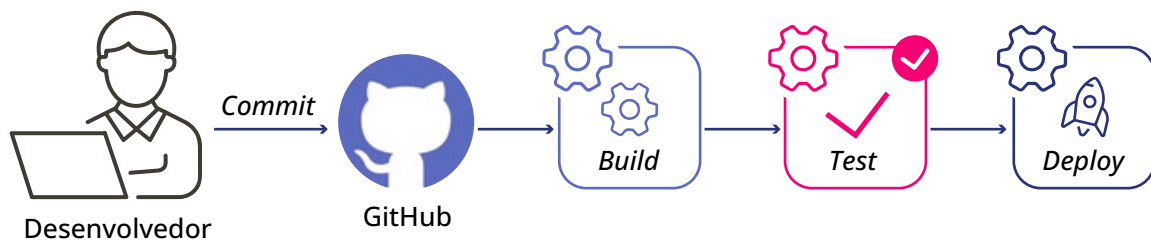


Figura 5 - Revisão e teste de código no GitHub

Fonte - Da autora (2025)

Regras de proteção de ramificações

Por fim, o ecossistema oferece ferramentas de segurança para impor boas práticas. A funcionalidade de regras de proteção de ramificação (*branch protection rules*) permite que os líderes técnicos configurem restrições rígidas nas ramificações críticas como a principal.

É possível, por exemplo, proibir tecnicamente que qualquer pessoa faça um *push* direto na *main*, obrigando o uso de PRs. Além disso, pode-se exigir que todo PR tenha a aprovação de pelo menos um ou dois revisores antes do *merge* e que todos os testes da CI estejam verdes.

Outra camada de segurança é o arquivo *CODEOWNERS*, que define quem são os donos de cada parte do sistema. Assim, se alguém alterar um arquivo na pasta **"/financeiro"**, o sistema automaticamente exige a revisão do time de finanças. Essas travas sistêmicas garantem que a qualidade e a segurança não dependam da memória ou da boa vontade das pessoas, mas sejam garantidas pelo próprio processo da plataforma.



O GitHub transformou o Git de uma ferramenta de linha de comando em uma plataforma de gestão de engenharia. Você aprendeu que a colaboração estruturada depende de uma topologia de remotos clara, do uso disciplinado dos PRs como portão de qualidade e da prática humana de revisão para disseminação de conhecimento. Somado à automação da CI, é criado um ambiente que permite velocidade com segurança.

PraticAI: Do código legado ao versionamento colaborativo

Hora de colocar a mão na massa! Pegue um script ou projeto antigo seu que está perdido em uma pasta local. Seu desafio prático tem três etapas.

Primeiro, inicialize o repositório local via terminal. Crie um arquivo README.md bem estruturado explicando o que esse projeto faz. Lembre-se de que uma documentação limpa e profissional é o verdadeiro cartão de visitas para qualquer portfólio de dados ou de desenvolvimento. Adicione os arquivos, faça seu primeiro *commit* na *main* e envie (*push*) para um novo repositório no seu GitHub.

Segundo, fuja do antipadrão de editar tudo direto na linha principal. No terminal, crie uma nova ramificação chamada **feature/melhoria-codigo**, adicione uma pequena funcionalidade ou refatore um trecho, faça o *commit* e suba essa ramificação para o servidor.

Por fim, abra o seu repositório no navegador e crie um *pull request* dessa sua nova funcionalidade contra a *main*. Observe a tela de **diff** (diferença) que a plataforma gera. É exatamente essa tela que a sua equipe técnica vai analisar durante uma revisão de código na vida real.

Agora que você já tem um código versionado, organizado e validado, é hora de aprender como gerenciar o fluxo de trabalho das pessoas que o produzem, usando ferramentas e processos ágeis como o *scrum* e *kanban*.



ENGENHARIA ÁGIL

O software não é escrito por máquinas, mas por pessoas, e a forma como o trabalho humano é organizado é tão crítica para o sucesso do produto quanto a escolha da linguagem de programação. Então, se o Git gerencia o código, as metodologias ágeis gerenciam o fluxo de valor.

Neste capítulo final, serão abordadas metodologias ágeis, que são processos fundamentados na redução de risco e na otimização de fluxo. Você vai entender como ferramentas como *scrum* e *kanban* se conectam diretamente com o PR e o DC estudados anteriormente, e como a estrutura do seu time pode ditar a arquitetura do seu software.

Do modelo cascata ao ágil

Historicamente, a engenharia de software tentou imitar a engenharia civil por meio do modelo cascata, no qual a premissa é simples e linear: definem-se todos os requisitos, desenha-se toda a arquitetura, codifica-se tudo e, só no final, testa-se e entrega-se ao cliente. No entanto, esse modelo falha no software porque ignora a volatilidade do problema (Sutherland, 2014). Diferente de uma ponte, em que a física é constante, no software os requisitos mudam enquanto o produto é construído.

O modelo ágil inverte essa lógica de risco. Em vez de uma grande entrega arriscada no final de um ano, pequenas entregas incrementais são feitas a cada duas semanas. Isso transforma um projeto gigante em uma série de pequenas apostas controladas, permitindo corrigir a rota rapidamente se o mercado mudar ou se a tecnologia falhar.

Essa mudança também exige uma nova mentalidade em relação à qualidade, conhecida como teste *'shift-left'*. No modelo antigo, testes eram realizados no final do projeto (à direita na linha do tempo). No modelo ágil, essas atividades são movidas para a esquerda, ou seja, para o início do desenvolvimento. Testes unitários são escritos antes ou durante a codificação, e a segurança é verificada a cada *commit*, não apenas na véspera do lançamento.



Curiosidade curiosa

Imagine o desenvolvimento de um aplicativo bancário. No modelo tradicional, uma vulnerabilidade que permitisse “transferências de valores negativos” (roubando dinheiro do banco) poderia passar despercebida até a fase de testes de aceitação, semanas após a codificação. Corrigi-la nesse estágio exigiria reabrir o chamado, alocar desenvolvedores, reescrever a lógica, passar por novos testes de regressão e realizar um novo *deploy*. O custo seria astronômico.

Com a abordagem *shift-left*, o próprio desenvolvedor escreve um teste automatizado antes mesmo de finalizar a funcionalidade. Ao tentar enviar o código, o teste roda localmente e falha instantaneamente (**`assert transfer(-50) == Error`**), alertando o engenheiro. O erro é corrigido em minutos, custando apenas alguns centavos de hora-trabalho e zero impacto no cronograma.



Métricas de engenharia: como medir o sucesso?

A pesquisa científica de Forsgren, Humble e Kim (2018) definiu as métricas DORA como o padrão ouro para medir a performance de engenharia de software. As quatro métricas fundamentais são:

1. **Lead time for changes:** Quanto tempo leva desde o primeiro *commit* até o código rodar em produção.
2. **Deployment frequency:** Com que frequência fazemos *deploy* bem-sucedido.
3. **Mean time to restore (MTTR):** Quanto tempo demoramos para corrigir uma falha em produção.
4. **Change failure rate:** Qual a porcentagem de *deploys* que causam falhas e exigem *rollback*.

Essas métricas provam matematicamente que velocidade e estabilidade não são opostos. Times de elite usam desenvolvimento *trunk-based*, CI/CD robusto e limites de WIP no *kanban* para fazer *deploys* múltiplos por dia com taxas de falha mínimas. O objetivo final de toda a gestão de código e agilidade que você estudou é otimizar esses números: entregar valor ao usuário o mais rápido possível, com a maior segurança possível.

Scrum

O *scrum* é frequentemente mal interpretado como apenas “fazer reuniões diárias”, mas, na verdade, é um *framework* rigoroso de feedback empírico desenhado para lidar com a complexidade. Ele divide o tempo em caixas fixas chamadas *sprints* (geralmente de duas semanas), que funcionam como mini projetos completos.

No início, há o planejamento (*sprint planning*), momento em que o time seleciona as tarefas do *backlog* (a lista de desejos do produto) que se compromete a entregar. Durante a execução, o *scrum* diário não serve para reportar status ao chefe, mas para que os engenheiros sincronizem o trabalho técnico: “O meu PR está bloqueando o seu? A API que estou construindo mudou?”.

Ao final, é feita a revisão (em que são feitas demonstrações com o software funcionando, não apenas em slides) e a retrospectiva, que é o momento de engenharia de processo: o time analisa suas métricas e decide, por exemplo, automatizar um teste que está lento ou alterar a política de ramificações para reduzir conflitos.

Um desafio constante gerenciado pelo *scrum* é a dívida técnica. O termo, cunhado por Ward Cunningham (1992), refere-se ao custo implícito de retrabalho causado pela escolha de uma solução fácil e rápida agora, em vez de usar uma abordagem melhor que levaria mais tempo.

Assim como uma dívida financeira, se não for paga (através de refatoração), os “juros” (complexidade do código) se acumulam até tornar o desenvolvimento insustentável. O *scrum* permite pagar essa dívida alocando tempo nas *sprints* para melhoria de código, e não apenas para novas funcionalidades.

Um conceito vital para engenheiros dentro do *scrum* é a *definition of done* (DoD). Uma tarefa não está “feita” quando o código foi escrito na máquina do desenvolvedor. Ela só está feita quando passou pela revisão do código, foi aprovada na CI, teve seus testes automatizados executados e está pronta para ser implantada em produção.

A DoD é o contrato de qualidade que impede o acúmulo de dívida técnica oculta e conecta diretamente o processo ágil com as ferramentas do GitHub sobre as quais você aprendeu anteriormente.

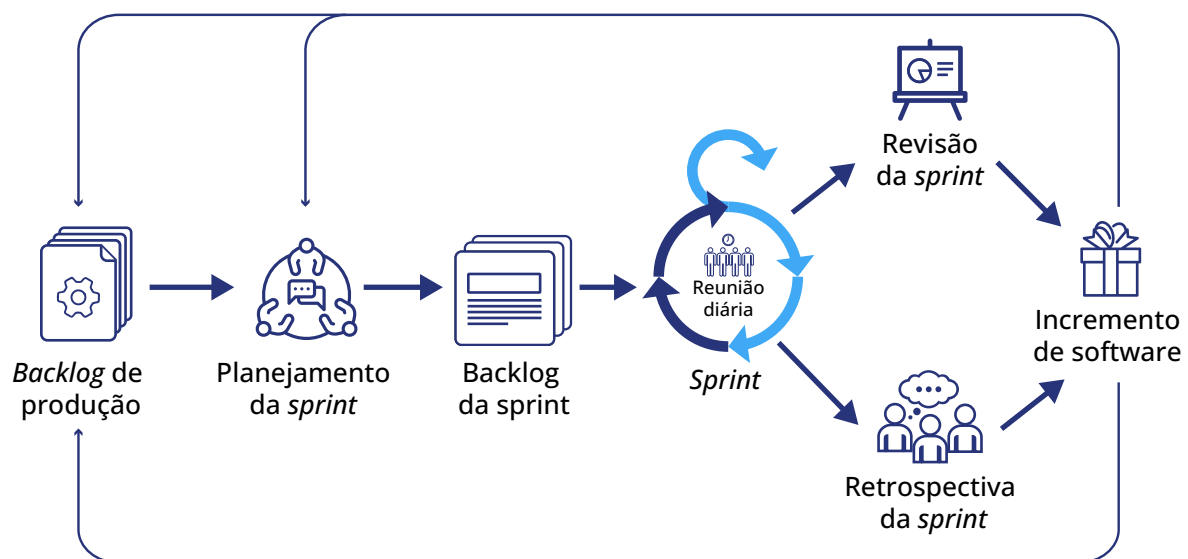


Figura 6 - Processo de *scrum* no desenvolvimento de programas
Fonte - Da autora (2025)

Kanban

Enquanto o *scrum* funciona em ciclos rítmicos, o *kanban* é focado no fluxo contínuo. Baseado no Sistema Toyota de Produção e adaptado para software por David J. Anderson (2010), o *kanban* não prescreve papéis ou reuniões, mas impõe uma regra física implacável: o limite de WIP (*work in progress*).



Atenção

A intuição nos diz que, para produzir mais, deve-se começar mais tarefas. A engenharia, porém, prova o contrário através da Lei de Little: quanto mais itens você tiver em progresso simultaneamente, maior será o tempo de espera (*lead time*) de cada um.

Desse modo, o quadro *kanban* não é apenas visualização, mas um sistema de controle de tráfego. Além do limite de WIP, Anderson (2010) também introduz o conceito técnico de classes de serviço, que define como diferentes tipos de trabalho são tratados na fila. Sendo eles:

- **Expedição (*expedite*):** É o trabalho que pode “furar fila”, geralmente voltado para bugs críticos de produção.
- **Data fixa (*fixed date*):** Tarefas que perdem valor se não forem entregues até certo dia, como uma funcionalidade para a Black Friday, por exemplo.
- **Padrão (*standard*):** Engloba a maioria das tarefas processadas por ordem de chegada.
- **Intangível (*intangible*):** Tarefas de baixo risco imediato, mas de alto valor a longo prazo, pois, se ignoradas, viram dívida técnica, como a refatoração.



Curiosidade curiosa

Imagine que uma equipe de sustentação de software estava sobrecarregada, com cada desenvolvedor lidando com cinco tarefas simultâneas. O tempo médio para resolver um chamado era de 15 dias. Ao aplicar o *kanban*, a gestão impôs um limite radical de WIP: máximo de uma tarefa por pessoa. Intuitivamente, parecia que a produtividade cairia. Na prática, esse *lead time* caiu para três dias. Afinal, o custo de troca de contexto (*context switching*) foi eliminado. Segundo estudos de Gerald Weinberg (1992), a cada tarefa extra adicionada, a eficiência cognitiva cai cerca de 20%. Com cinco tarefas, o desenvolvedor passa mais tempo tentando lembrar onde parou do que realmente codificando.

Outro conceito trazido da Toyota é o *andon cord*. Na fábrica, qualquer operário pode puxar uma corda para parar a linha de montagem inteira se encontrar um defeito. No software moderno, isso é implementado pelo pipeline de CI/CD. Se um teste falha ou uma vulnerabilidade é detectada, o pipeline “puxa a corda”, bloqueando o *deploy* para todos. A produção para até que o erro seja corrigido. Isso garante que defeitos nunca avancem para a próxima etapa, reduzindo drasticamente o custo de correção.

A ilustração a seguir demonstra como esse fluxo contínuo é organizado visualmente. Note as colunas delimitadas e, crucialmente, os limites de WIP (números no topo) que impedem o time de puxar mais trabalho do que é capaz de processar.

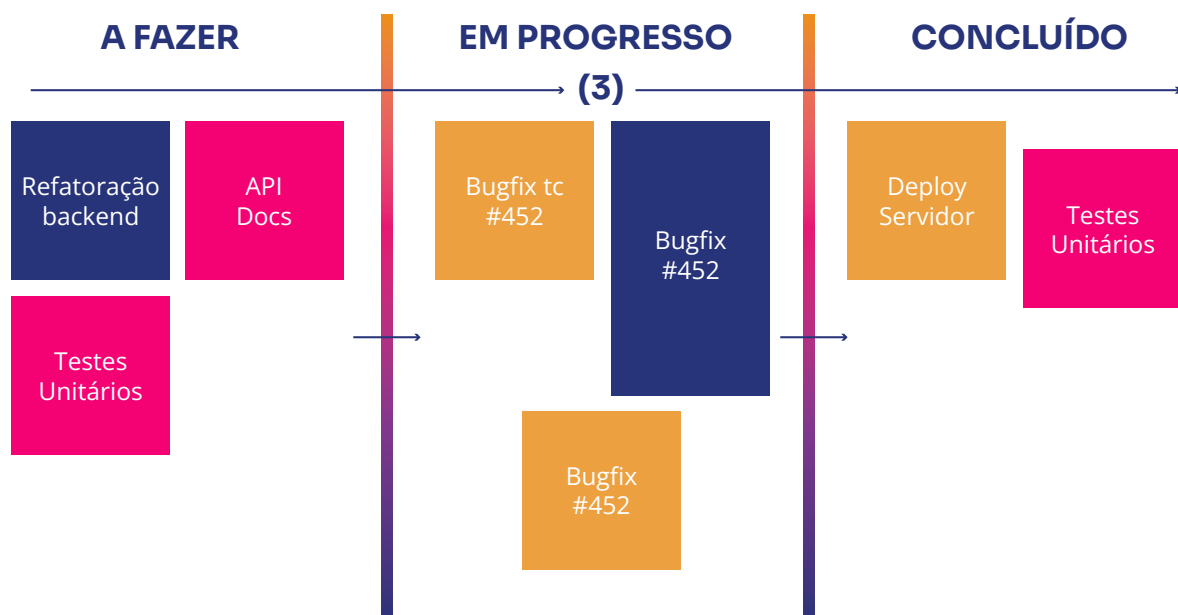


Figura 7 - Exemplo de *kanban*
Fonte - Da autora (2025)

Lei de Conway: a estrutura do time define a arquitetura

Ao organizar times ágeis, um fenômeno sociotécnico chamado Lei de Conway deve ser considerado. Melvin Conway postulou em 1967 que qualquer organização que projeta um sistema produzirá um design cuja estrutura é uma cópia da estrutura de comunicação da organização.

Isso significa que, se você tem um time de 100 pessoas em que todos falam com todos, você inevitavelmente produzirá um software monolítico e acoplado. Assim, se você deseja uma arquitetura moderna de serviços pequenos e independentes, você deve primeiro quebrar sua organização em times pequenos e autônomos, chamados de *squads* (ou *two-pizza teams*, na Amazon).



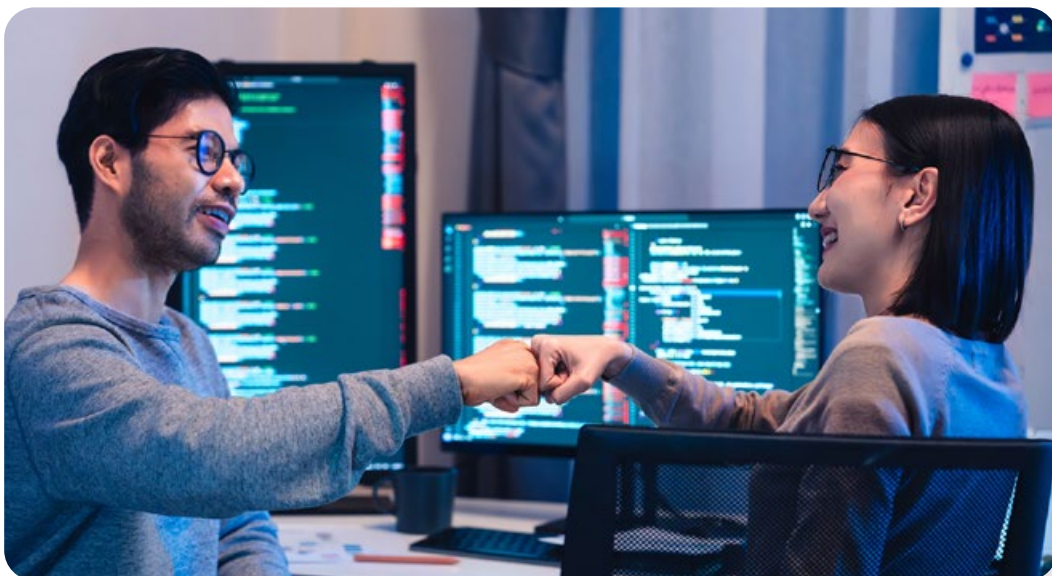
Dica

O Git e o CI/CD facilitam isso, permitindo que cada *squad* tenha seu próprio repositório e pipeline de *deploy*, sem depender de uma coordenação central massiva.

Três vias do DevOps

Para conectar o desenvolvimento ágil (Dev) com a estabilidade da operação (Ops), Kim *et al.* (2016) propõe o modelo das “três vias” que serve como a filosofia de engenharia por trás das ferramentas, segundo o qual:

1. **Primeira via** (fluxo): Trata de acelerar a entrega da esquerda (desenvolvimento) para a direita (cliente), aplicando o CI/CD e a automação de testes para garantir que o código flua rápido e sem atrito.
2. **Segunda via** (feedback): Foca em amplificar o retorno da direita para a esquerda através de telemetria e monitoramento. Assim, não basta só lançar o código, é preciso saber imediatamente – por meio de *logs* e alertas – se um erro ocorreu, antes do suporte ser acionado.
3. **Terceira via** (aprendizado contínuo): Promove uma cultura de experimentação e a prática de pós-mortems sem culpa, de modo que, ao corrigir uma falha, o foco seja melhorar o processo, não punir o indivíduo.



PraticAI: Limitando o WIP na vida real

A agilidade e o fluxo contínuo não servem apenas para linhas de código. Escolha um projeto técnico ou acadêmico que você está desenvolvendo agora. Em uma ferramenta digital ou em um quadro físico, crie três colunas básicas:

A Fazer, Em Progresso e Concluído.

Liste todas as suas tarefas pendentes na primeira coluna. Agora, aplique a regra de ouro da engenharia de fluxo: estabeleça um limite de WIP (*work in progress*) de no máximo 2 para a sua coluna “Em Progresso”.

Sempre que você sentir a tentação (o anti-padrão) de começar uma terceira tarefa simultânea para “adiantar as coisas”, você deve se obrigar a parar e finalizar uma das duas que já estão em andamento para liberar espaço.

Durante uma semana, observe como, paradoxalmente, focar em menos coisas ao mesmo tempo fará você entregar resultados com muito mais velocidade e qualidade.

O fluxo unificado: GitFlow + scrum + CI/CD

A verdadeira potência surge quando todas as pontas são conectadas em um fluxo de trabalho diário, palpável e coerente, conhecido na indústria como DevOps. Para visualizar isso na prática, imagine a rotina de desenvolvimento de uma equipe em um projeto real.

Tudo começa com uma necessidade de negócio no backlog do *scrum*, como, por exemplo, a criação de uma nova base de dados para alimentar um dashboard. Após o alinhamento na reunião diária, o desenvolvedor identifica sua prioridade e puxa essa tarefa para a *sprint*, movendo o cartão correspondente para a coluna “Em Progresso” do *kanban*.

É vital notar um antipadrão clássico neste momento: puxar vários cartões simultaneamente para tentar demonstrar produtividade. Na realidade, isso viola o limite de WIP do *kanban*, pulveriza o foco e gera gargalos na esteira.

Com a tarefa definida no quadro ágil, a ação passa para a arquitetura de versionamento. Antes de escrever a primeira linha de código, o desenvolvedor garante que sua máquina está sincronizada com a versão mais recente do projeto e cria uma ramificação de funcionalidade isolada:



Garante que está na ramificação de integração e a atualiza

```
git checkout develop
```

```
git pull origin develop
```

Cria e já entra na nova ramificação da tarefa

```
git checkout -b feature/base-looker-studio
```

Conforme constrói a solução, ele realiza pequenos *commits* com mensagens semânticas e descritivas. Isso permite que o progresso seja rastreável e seguro:



Prepara o arquivo modificado para o snapshot

```
git add query_vendas.sql
```

Salva a alteração com uma mensagem clara (padrão Conventional Commits)

```
git commit -m "feat: implementa extração de dados para o Looker Studio"
```

O antipadrão a ser evitado aqui é o do “desenvolvedor solitário”, que programa por dias a fio e faz um único *commit* gigante na sexta-feira com a mensagem “ajustes finais”, ou pior, altera diretamente a ramificação principal. Uma arquitetura de histórico bagunçada assim destrói o trabalho colaborativo e impossibilita que colegas depurem o código em caso de falhas.

Quando a funcionalidade está pronta localmente, ela é enviada ao servidor para iniciar o processo de revisão:



Envia a funcionalidade para o repositório remoto

```
git push origin feature/base-looker-studio
```



Esse envio permite a abertura de um *pull request* (PR) no GitHub. Esse ato dispara automaticamente o pipeline de CI, que atua como um *andon cord* digital rodando testes automatizados. Se o robô identificar problemas, o PR é bloqueado. Se tudo estiver verde, o código passa pela revisão humana do time, onde o foco é a lógica e a qualidade arquitetural.

Somente após a aprovação e a fusão (*merge*) segura deste código na ramificação *develop* (ou *main*) é que o cartão no *kanban* é finalmente movido para a coluna “Concluído”. Perceba como não há fronteiras rígidas: o processo humano e ágil do *scrum* é materializado e protegido o tempo todo pelas ferramentas de versão (Git) e de automação (CI/CD).



Chegamos ao fim da nossa jornada pela engenharia do desenvolvimento moderno. Você começou entendendo a integridade matemática do Git, passando pela arquitetura das estratégias de ramificação, a colaboração segura no GitHub e finalizando com a mentalidade ágil do *scrum* e *kanban*.

O desenvolvimento de software profissional é um ato de coordenação. As ferramentas (Git) e os processos (*scrum*) existem para resolver o problema fundamental da entropia em sistemas complexos.

Espero que este material tenha lhe fornecido não apenas os comandos, mas a visão sistêmica necessária para atuar em times de alta performance, em que o código flui como um rio, limpo, testado e constante, gerando valor real para o mundo. Até a próxima!



REFERÊNCIAS

ANDERSON, D. J. **Kanban**: successful evolutionary change for your technology business. Sequim: Blue Hole Press, 2010.

CHACON, S.; STRAUB, B. **Pro Git**. 2. ed. Berkeley: Apress, 2014.

CONWAY, M. E. How do committees invent?. **Datamation**, p. 28-31, 1968. Disponível em: <https://www.melconway.com/Home/pdf/committees.pdf>. Acesso em: 6 jan. 2025.

CUNNINGHAM, W. The WyCash Portfolio Management System. *In*: OOPSLA '92, 1992, Vancouver. **Addendum to the Proceedings** [...]. Vancouver: ACM, 1992. p. 29-30. Experience Report. Disponível em: <https://doi.org/10.1145/157710.157715>. Acesso em: 6 jan. 2025.

DRIESSEN, V. A successful Git branching model. **nvie.com**, 5 jan. 2010. Disponível em: <https://nvie.com/posts/a-successful-git-branching-model/>. Acesso em: 6 jan. 2025.

FORSGREN, N.; HUMBLE, J.; KIM, G. **Accelerate**: the science of lean software and DevOps. Portland: IT Revolution Press, 2018.

GOOGLE. Google engineering practices documentation. **Google**, [202-]. Disponível em: <https://google.github.io/eng-practices/> Acesso em: 5 dez. 2025.

HUMBLE, J.; FARLEY, D. **Continuous delivery**: reliable software releases through build, test, and deployment automation. White Plains: Addison-Wesley, 2010.

KIM, G. *et al.* **The DevOps Handbook**. Portland: IT Revolution Press, 2016.

SUTHERLAND, J. **Scrum**: the art of doing twice the work in half the time. New York: Crown Business, 2014.

WEINBERG, G. M. **Quality software management**: systems thinking. New York: Dorset House, 1992.



SCTEC